

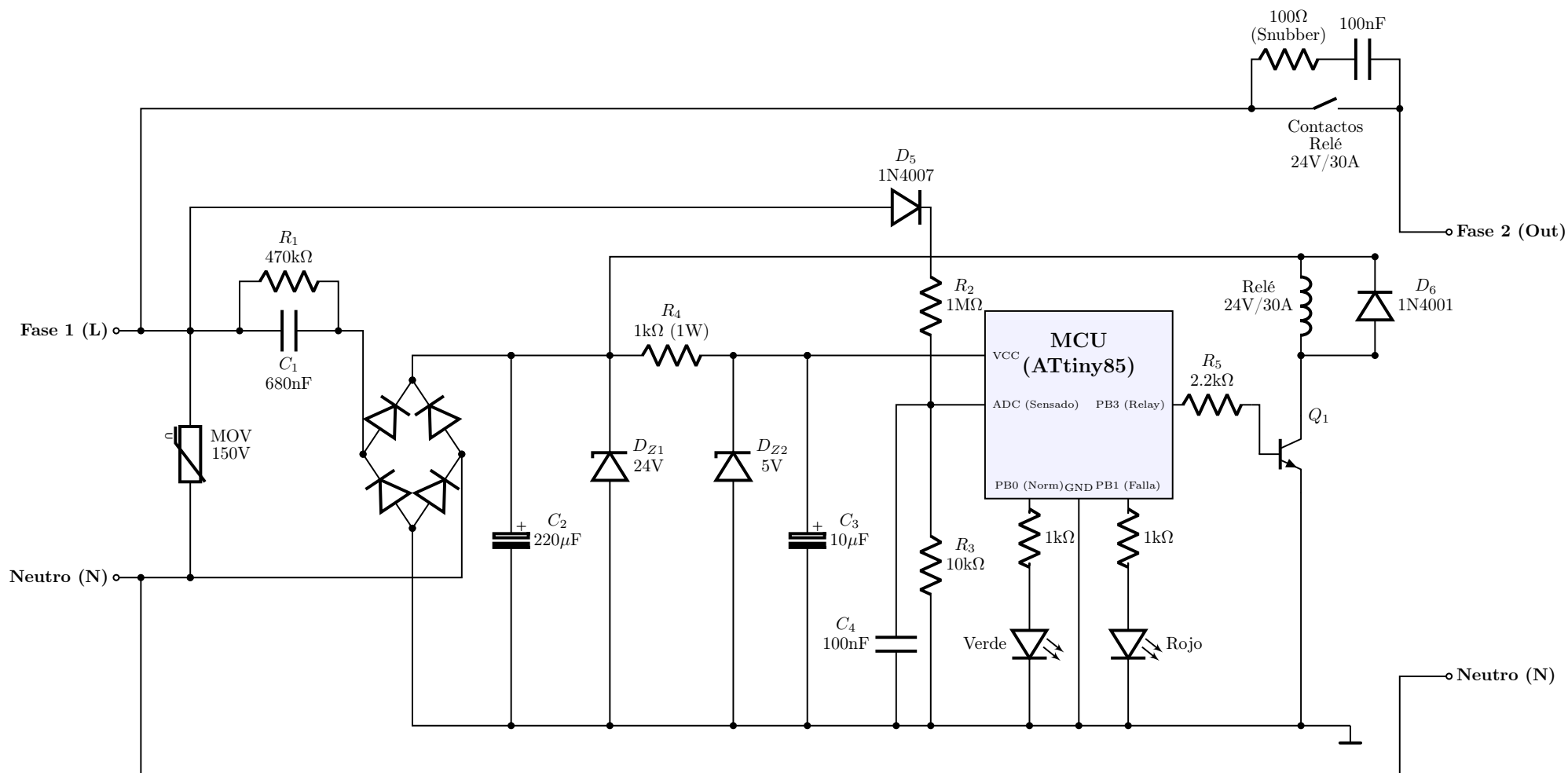


Figura 1: Esquema Mejorado V2.0: Protector de Voltaje 120VAC controlado por Microcontrolador (MCU).

Respuesta del Circuito ante una Subida de Tensión

Cuando ocurre una subida de tensión (sobretensión) en la entrada de la **Fase 1**, el circuito actúa en dos niveles de protección de forma automática:

1. **Protección contra Transitorios (Picos ultrarrápidos):** Si la subida es un pico transitorio muy rápido y de alta tensión (por ejemplo, inducido por un rayo, conexión de grandes cargas inductivas o ruido severo de la red), el **Varistor (MOV de 150V)** entra en acción. Al superar su tensión de umbral, su resistencia interna cae drásticamente, absorbiendo la energía del pico y derivando el exceso directamente hacia el Neutro. Esto evita que la espiga destructiva llegue al resto del circuito interno y a la carga acoplada.
2. **Protección contra Sobretenensión Sostenida (Corte activo):** Si el voltaje RMS de la red se eleva de manera sostenida por encima del límite seguro de funcionamiento, el sistema actúa aislando físicamente la carga:
 - **Detección:** El divisor de tensión formado por R_2 (1M Ω) y R_3 (10k Ω), junto con la rectificación de media onda de D_5 y el filtrado de C_4 , entregan una pequeña muestra de voltaje DC directamente al pin analógico (**ADC**) del MCU. Esta muestra es proporcional al voltaje real instantáneo de la Fase 1.
 - **Decisión del MCU:** El microcontrolador (**ATtiny85**) lee y evalúa de forma continua esta conversión ADC. Al detectar que el valor sobrepasa el umbral máximo preprogramado en su software, el MCU entra en estado de "falla por sobretenensión".
 - **Aislamiento de la carga:** El MCU apaga la salida **PB3** (bajándola a 0V). Esto retira la polarización de la base del transistor Q_1 , lo cual desenergiza instantáneamente la bobina del **Relé**. Los contactos del relé se abren, desconectando de inmediato la **Fase 2 (Out)** y salvaguardando al equipo que esté enchufado al protector.
 - **Indicadores Visuales:** Para alertar al usuario de la condición anormal, el MCU apaga el LED Verde (estado normal en **PB0**) y enciende el LED Rojo (estado de falla en **PB1**).

Recuperación: Tras el corte, el microcontrolador seguirá monitoreando el voltaje en Fase 1. Una vez que la tensión regrese y se estabilice en niveles seguros de operación, el MCU típicamente iniciará un temporizador de retardo (por ejemplo, 3 minutos) antes de volver a energizar el relé y encender el LED verde. Este retardo es crucial para permitir la descompresión en motores y proteger equipos sensibles, como los compresores de los refrigeradores.

Código de Programación (Arduino/C++)

El siguiente código implementa la lógica de control para el microcontrolador ATtiny85. Se encarga de leer el voltaje rectificado, evaluar los umbrales de seguridad y accionar el relé y los indicadores LED, incluyendo un temporizador de recuperación sin bloqueos.

```
1 /*
2  * PROTECTOR DE VOLTAJE 120VAC - V2.0
3  * Microcontrolador: ATtiny85
4  * Reloj interno: 1MHz o 8MHz
```

```

5  */
6
7  // ===== DEFINICIÓN DE PINES =====
8  const int PIN_LED_NORM = 0; // PBO - LED Verde (Voltaje Normal)
9  const int PIN_LED_FAIL = 1; // PB1 - LED Rojo (Falla por sobre/bajo voltaje)
10 const int PIN_SENSADO = A1; // PB2 - Entrada Analógica ADC (Sensado de voltaje)
11 const int PIN_RELAY = 3;    // PB3 - Control del Relé
12
13 // ===== CONSTANTES Y UMBRALES =====
14 /*
15  * Los umbrales ADC van de 0 a 1023 (equivalente de 0V a 5V en el pin).
16  * Deberás calibrar estos valores empíricamente según tu divisor de tensión
17  * (1M ohm y 10k ohm) y la caída en el diodo rectificador.
18  *
19  * Por ejemplo, valores hipotéticos:
20  */
21 const int UMBRAL_MAX = 750; // Equivalente aprox. a > 135V RMS
22 const int UMBRAL_MIN = 400; // Equivalente aprox. a < 90V RMS
23
24 // Tiempo de espera de recuperación (en milisegundos)
25 // 3 minutos = 3 * 60 * 1000 = 180000 ms
26 const unsigned long TIEMPO_RECUPERACION = 180000UL;
27
28 // ===== VARIABLES GLOBALES =====
29 bool estadoFalla = true; // Iniciamos en estado de falla por seguridad
30 unsigned long tiempoInicioEstable; // Para llevar la cuenta regresiva sin usar delay()
31
32 void setup() {
33     // Configuración de los pines de salida
34     pinMode(PIN_RELAY, OUTPUT);
35     pinMode(PIN_LED_NORM, OUTPUT);
36     pinMode(PIN_LED_FAIL, OUTPUT);
37
38     // Configuración del pin analógico
39     pinMode(PIN_SENSADO, INPUT);
40
41     // Estado inicial de seguridad: Carga desconectada
42     digitalWrite(PIN_RELAY, LOW);
43     digitalWrite(PIN_LED_NORM, LOW);
44     digitalWrite(PIN_LED_FAIL, HIGH); // LED rojo encendido
45
46     // Iniciamos el temporizador

```

```

47  tiempoInicioEstable = millis();
48  }
49
50  void loop() {
51      // 1. Leer voltaje de la red rectificado
52      int lecturaADC = analogRead(PIN_SENSADO);
53
54      // 2. Evaluar la lectura contra los límites de seguridad
55      if (lecturaADC > UMBRAL_MAX || lecturaADC < UMBRAL_MIN) {
56          // --- CONDICIÓN DE FALLA ---
57          estadoFalla = true;
58
59          // Al ocurrir o mantenerse la falla, reiniciamos continuamente el cronómetro
60          tiempoInicioEstable = millis();
61
62          // Acciones de aislamiento inmediato
63          digitalWrite(PIN_RELAY, LOW);
64          digitalWrite(PIN_LED_NORM, LOW);
65          digitalWrite(PIN_LED_FAIL, HIGH);
66
67      } else {
68          // --- VOLTAJE DENTRO DE RANGO NORMAL ---
69          // Si estábamos en falla, evaluamos si ya pasó el tiempo seguro
70          if (estadoFalla) {
71              if (millis() - tiempoInicioEstable >= TIEMPO_RECUPERACION) {
72                  estadoFalla = false;           // Ha superado el tiempo de seguridad
73                  digitalWrite(PIN_RELAY, HIGH); // Encender Relé (Acoplar carga)
74                  digitalWrite(PIN_LED_FAIL, LOW); // Apagar LED de falla
75                  digitalWrite(PIN_LED_NORM, HIGH); // Encender LED Normal
76              }
77          }
78      }
79  }

```

Notas sobre el código

- **Uso de millis() en lugar de delay():** Fíjate que al calcular el tiempo de retardo no utilizamos bloqueos del procesador. Esto permite que el ATtiny85 siga leyendo el voltaje miles de veces por segundo durante los 3 minutos de espera; si en el minuto 2 ocurre otro pico de voltaje, el temporizador se reinicia inmediatamente a 0.
- **Calibración Analógica:** Se han puesto 750 y 400 como valores simulados (para UMBRAL_MAX y UMBRAL_MIN). Dado que el divisor de tensión

real ($1\text{M}\Omega$ y $10\text{k}\Omega$) y la caída de 0.7V en el diodo 1N4007 introducen variaciones, deberás inyectar un voltaje conocido en el circuito y medir qué número real escupe el `analogRead()` para afinar estas constantes antes de pasarlo a producción.